

Real-world (one reusable session) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — a module-level session

```
# session_reuse_practice.py
import requests

session = requests.Session()
session.headers.update({"Authorization": "Bearer fake-token"})

urls = [
    "https://api.github.com",
    "https://api.github.com/zen",
    "https://api.github.com/octocat",
]

for url in urls:
    response = session.get(url)
    print(url, "->", response.status_code)

print("Authorization header set to:", session.headers["Authorization"])
```

Expected output:

```
https://api.github.com -> 200
https://api.github.com/zen -> 200
https://api.github.com/octocat -> 200
Authorization header set to: Bearer fake-token
```

This works and proves the header sticks across calls. It doesn't yet confirm the header was actually *sent* on the wire, just that it's set on the session object — the Intermediate version checks that distinction.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a typed factory function

```
# session_reuse_practice.py
import requests

def make_authenticated_session(token: str) -> requests.Session:
    session = requests.Session()
    session.headers.update({"Authorization": f"Bearer {token}"})
    return session

def main() -> None:
    session = make_authenticated_session("fake-token")

    urls = [
        "https://api.github.com",
        "https://api.github.com/zen",
        "https://api.github.com/octocat",
    ]

    for url in urls:
        response = session.get(url)
        print(f"{url} -> {response.status_code}")

    sent_header = response.request.headers.get("Authorization")
    print(f"header actually sent on the last request: {sent_header}")

if __name__ == "__main__":
    main()
```

Expected output:

```
https://api.github.com -> 200
https://api.github.com/zen -> 200
```

```
https://api.github.com/octocat -> 200  
header actually sent on the last request: Bearer fake-token
```

`response.request.headers` reads back the *actual* request object `requests` sent, not the session's own dict — confirming the header genuinely went out over the wire, not just that it's set somewhere in Python.

Difference from Basic: a named, typed factory function (`make_authenticated_session`) instead of module-level statements is reusable and importable, and checking `response.request.headers` instead of `session.headers` confirms the header was actually transmitted, not just configured.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a class that owns the session, headers set once and never touched again

```
# session_reuse_practice.py
import requests

class ApiClient:
    def __init__(self, base_url: str, token: str) -> None:
        self.base_url = base_url
        self.session = requests.Session()
        self.session.headers.update({"Authorization": f"Bearer {token}"})
        # headers are configured here, once, at construction – nothing
        # later in this class (or any caller) should mutate them again

    def get(self, path: str, **extra_headers: str) -> requests.Response:
        # per-call headers merge on top for this one request only,
        # without ever touching self.session.headers
        return self.session.get(f"{self.base_url}{path}", headers=extra_headers)

    def close(self) -> None:
        self.session.close()

def main() -> None:
    client = ApiClient(base_url="https://api.github.com", token="fake-token")
    try:
        for path in ["", "/zen", "/octocat"]:
            response = client.get(path)
            print(f"{path or '/'} -> {response.status_code}")
    finally:
        client.close()

if __name__ == "__main__":
    main()
```

Expected output:

```
-> 200
/zen -> 200
/octocat -> 200
```

`client.close()` in a `finally:` block closes the session's pooled connections even if one of the calls above raised — the same reasoning as `with open(path) as f:` for files.

Approach 2 — the same class, used as a context manager

```
# session_reuse_practice.py
import requests

class ApiClient:
    def __init__(self, base_url: str, token: str) -> None:
        self.base_url = base_url
        self.session = requests.Session()
        self.session.headers.update({"Authorization": f"Bearer {token}"})

    def get(self, path: str) -> requests.Response:
        return self.session.get(f"{self.base_url}{path}")

    def __enter__(self) -> "ApiClient":
        return self

    def __exit__(self, *exc_info: object) -> None:
        self.session.close()

with ApiClient(base_url="https://api.github.com", token="fake-token") as client:
    response = client.get("/zen")
    print(response.status_code)
```

Expected output:

```
200
```

`__enter__` / `__exit__` makes `ApiClient` usable in a `with` block, so the connection cleanup happens automatically at the end of the block, the same way `with requests.Session() as session:` works for a bare session — no separate `try/finally` needed at the call site.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's factory function returns a session that's still a module-level object anyone importing it could mutate from anywhere — fine for a script, risky in a bigger program with multiple call sites or threads. Both

Advanced approaches fix that by keeping the session as private internal state of a class, configured once at construction, with headers never exposed for later mutation — the "configure once, read-only after" rule from Hint 1's Advanced section, enforced by the class's shape instead of just by discipline. Approach 2 additionally makes cleanup automatic via `with`, instead of relying on a caller remembering to call `.close()`.

Which one should you actually write? Intermediate's factory function is enough for a script or a one-off tool with a single, short-lived call site. Approach 1 (or Approach 2, if you like the `with` syntax) is what you actually want the moment this client is imported and used from more than one place — which is exactly what happens in this document's Build Task, and again in Doc04's `chat_client.py`. Bundling the `base_url` and session together, and never exposing a way to mutate the session's headers after construction, removes an entire category of "why did this one call suddenly have the wrong header" bug before it can happen.